

Criando um *iterador*

Todo `for` do Python conversa com um **iterador**: um objeto que entrega um item por vez e avisa quando acabou. Nesta aula: a diferença entre **iterável** e **iterador**, as funções `iter()` e `next()`, o que o `for` faz por baixo dos panos e como escrever o seu próprio iterador com `__iter__` e `__next__`.

Iterável × iterador

`iter()` · `next()`

`StopIteration`

O `for` por dentro

`__iter__` · `__next__`

Iterador se esgota

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Iterators; Glossário: iterable, iterator; Built-in Types: Iterator Types), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Um item *por vez*

protocolo.py

```
notas = [7, 9, 10]
it = iter(notas)          # pede um iterador à lista
print(type(it).__name__)
print(next(it))           # entrega um item por vez
print(next(it))
print(next(it))
try:
    next(it)              # acabou: não há próximo
except StopIteration:
    print("StopIteration: fim dos itens")
```

SAÍDA

```
list_iterator
7
9
10
StopIteration: fim dos itens
```

ITER(X)
pede o iterador



NEXT(IT)
próximo item



STOPITERATION
acabou

- ✓ `iter()` transforma um iterável num **iterador**
- ✓ Cada `next()` avança uma posição e não volta atrás
- ✓ No fim, o iterador lança `StopIteration`

Lista é iterável, *não iterator*

nao_iterador.py

```
notas = [7, 9, 10]
print(next(notas)) # lista é iterável, não iterator
```

SAÍDA

```
Traceback (most recent call last):
  File "nao_iterador.py", line 2, in <module>
    print(next(notas)) # lista é iterável, não iterator
          ~~~~~^^^^^^
TypeError: 'list' object is not an iterator
```

	ITERÁVEL	ITERADOR
O que tem	<code>__iter__</code>	<code>__iter__</code> e <code>__next__</code>
Exemplos	list, str, dict, range	o que <code>iter()</code> devolve
Aceita <code>next()</code> ?	Não	Sim
Percorre de novo?	Sim, quantas vezes quiser	Não: se esgota

Todo iterator também é iterável: o `__iter__` dele devolve ele mesmo.

O que o *for* faz por você

for_por_dentro.py

```
frutas = ["maçã", "uva", "kiwi"]

# O que o for faz por baixo dos panos:
it = iter(frutas)
while True:
    try:
        fruta = next(it)
    except StopIteration:
        break
    print(fruta)
```

SAÍDA

```
maçã
uva
kiwi
```

- ✓ O `for fruta in frutas` chama `iter(frutas)` uma vez
- ✓ A cada volta, chama `next()` e guarda o valor em `fruta`
- ✓ Quando vem `StopIteration`, o laço termina sem erro

Você não precisa escrever esse `while`: o `for` já faz. Ver por dentro explica por que qualquer objeto com o protocolo funciona num `for`.

Uma classe com `__iter__` e `__next__`

contagem.py

```
class Contagem:
    """Iterador: conta de n até 1."""

    def __init__(self, n):
        self.atual = n

    def __iter__(self):
        return self        # o iterador devolve a si mesmo

    def __next__(self):
        if self.atual < 1:
            raise StopIteration
        valor = self.atual
        self.atual -= 1
        return valor

if __name__ == "__main__":
    for x in Contagem(3):
        print(x)
    print(list(Contagem(5)))
```

- ✓ `__iter__` devolve `self`: o objeto é o próprio iterador
- ✓ `__next__` calcula o próximo valor e guarda o estado em `self.atual`
- ✓ Para encerrar, `raise StopIteration`
- ✓ Funciona com `for`, `list()`, `sum()`, `in` ...

SAÍDA

```
3
2
1
[5, 4, 3, 2, 1]
```

O iterador *se esgota*

esgota.py

```
from contagem import Contagem

c = Contagem(3)
print(list(c))    # primeira volta: consome tudo
print(list(c))    # segunda volta: o iterador já acabou
```

SAÍDA

```
[3, 2, 1]
[]
```

- ✓ O estado (`self.atual`) fica guardado no objeto
- ✓ Depois da primeira volta, ele já está no fim: a segunda volta não tem itens
- ✓ Para recomeçar, crie um iterador novo: `Contagem(3)` de novo

É um erro comum: guardar um iterador numa variável e tentar percorrê-lo duas vezes.

Um iterador *novo a cada for*

playlist.py

```
class Playlist:
    """Iterável: cada for ganha um iterador novo."""

    def __init__(self, *musicas):
        self.musicas = list(musicas)

    def __iter__(self):
        return iter(self.musicas)

p = Playlist("Intro", "Refrão", "Final")
print(list(p))
print(list(p))    # funciona de novo
print("Refrão" in p)
```

SAÍDA

```
['Intro', 'Refrão', 'Final']
['Intro', 'Refrão', 'Final']
True
```

- ✓ A `Playlist` é iterável: tem só `__iter__`
- ✓ Cada chamada de `__iter__` entrega um iterador novo (o da lista interna)
- ✓ Por isso dá para percorrer de novo, e o `in` funciona

Regra prática: quem guarda os dados é iterável; quem guarda a posição é o iterador.

O seu *próprio range*

Enunciado: crie a classe `Passos(inicio, fim, passo=1)`, um iterador que entrega os inteiros de `inicio` até antes de `fim`, pulando de `passo` em `passo`.

Uma resposta possível

```
class Passos:
    """Como range(inicio, fim, passo), só com inteiros."""

    def __init__(self, inicio, fim, passo=1):
        self.atual = inicio
        self.fim = fim
        self.passo = passo

    def __iter__(self):
        return self

    def __next__(self):
        if self.atual >= self.fim:
            raise StopIteration
        valor = self.atual
        self.atual += self.passo
        return valor

print(list(Passos(0, 10, 3)))
print(list(Passos(5, 8)))
print(list(Passos(4, 2)))
```

SAÍDA

```
[0, 3, 6, 9]
[5, 6, 7]
[]
```

- ✓ Quando `inicio >= fim`, a lista sai vazia, como no `range`
- ✓ Desafio: aceite passo negativo (contagem para trás)
- ✓ Desafio 2: transforme em iterável reutilizável, como a `Playlist`

Teste o que *ficou*

1. O que `next(iter([5, 6]))` devolve?

A) [5, 6]

B) 5

C) 6

D) Erro

B.

2. Como o iterador avisa que acabou?

A) Devolve None

B) Devolve False

C) Lança `StopIteration`

D) Recomeça do início

C.

3. Num iterador, o `__iter__` costuma devolver:

A) `self`

B) Uma lista

C) None

D) O próximo item

A.

4. `it = iter([1, 2]); list(it); list(it)` — a segunda lista é:

A) [1, 2]

B) [2]

C) []

D) Erro

C. O iterador já se esgotou.

Um item por vez, *até o fim*

Iteráveis guardam os dados; iteradores guardam a posição. `iter()` pede o iterador, `next()` avança e `StopIteration` encerra, e é isso que o `for` faz por você. Para criar o seu, escreva `__iter__` (devolvendo `self`) e `__next__`, e lembre que ele se esgota.

`iter()` · `next()`

`StopIteration`

`__iter__` · `__next__`

Iterável reutilizável

Próxima aula → 02 · Iteradores Default

Fontes: docs.python.org/3/tutorial/classes.html#iterators; docs.python.org/3/glossary.html (`iterable`, `iterator`); docs.python.org/3/library/stdtypes.html#iterator-types; docs.python.org/3/library/functions.html (`iter`, `next`), consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.